
Azure Hub-and-Spoke Employee Portal

Complete Deployment Runbook

Hub + P2S VPN + Azure Firewall + Windows VMSS/.NET/IIS + Linux/Flask + Microsoft Entra ID

Purpose: document the completed lab so another learner can understand, reproduce, validate, troubleshoot and safely remove it.

Final scope

- Region: Central India.
- Hub VNet 10.0.0.0/16 with only GatewaySubnet and AzureFirewallSubnet.
- Spoke 1: Windows Server 2022 VMSS with IIS and ASP.NET Core/.NET Employee Portal.
- Spoke 2: Ubuntu 22.04.5 LTS VM with Python Flask Employee Portal.
- Microsoft Entra ID authentication patterns for both applications; MFA occurs when Entra policy requires it.
- P2S VPN is the administrator access path.
- Excluded: on-premises migration, DR region, Bastion, management/shared-services subnets and private endpoints.

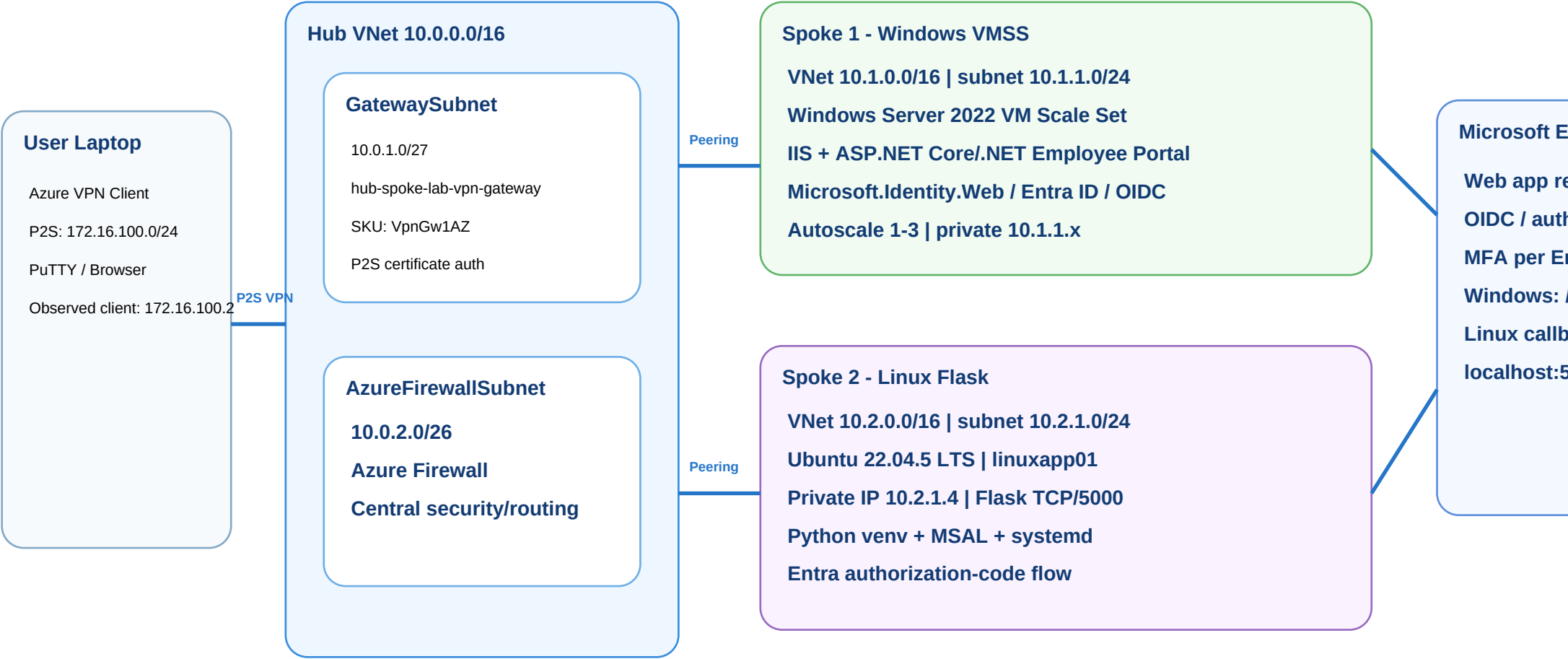
Version: 27 September 2026

Contents

1. Architecture and address plan
2. Azure service definitions
3. Prerequisites and tools
4. Terraform deployment workflow
5. P2S VPN
6. Firewall, routing and NSGs
7. Spoke 1 Windows VMSS/IIS/.NET
8. Spoke 1 Entra ID
9. Spoke 2 Ubuntu/Flask
10. PuTTY/SSH
11. Spoke 2 Entra ID/MSAL
12. Testing
13. Troubleshooting
14. Operations/security improvements
15. Cost control/teardown
16. Redeployment checklist
17. Command reference

Azure Hub-and-Spoke Architecture - Implemented Project

Central India | P2S VPN | Azure Firewall | Windows VMSS/.NET/IIS | Ubuntu/Flask | Microsoft Entra ID



Not in project: on-premises, DR, Bastion, management/shared-services subnets, private endpoints.

This page uses the implemented subnet ranges and should be preferred over earlier conceptual diagrams.

1. Architecture and Address Plan

The project uses a private hub-and-spoke topology. The Hub centralizes VPN connectivity and firewalling. Spoke 1 hosts the Windows VMSS workload; Spoke 2 hosts the Linux workload. VNet peering provides private Azure backbone connectivity.

Component	Implemented value	Role
Resource group	hub-spoke-lab-rg	Lab resource container
Hub VNet	hub-spoke-lab-hub-vnet 10.0.0.0/16	Central network
GatewaySubnet	10.0.1.0/27	Azure VPN Gateway
AzureFirewallSubnet	10.0.2.0/26	Azure Firewall
Spoke 1	10.1.0.0/16	Windows application spoke
vmss-subnet	10.1.1.0/24	Windows VMSS
Spoke 2	10.2.0.0/16	Linux application spoke
linux-app-subnet	10.2.1.0/24	Linux VM
Linux VM	linuxapp01 / 10.2.1.4	Flask app
P2S pool	172.16.100.0/24	VPN clients
VPN Gateway	hub-spoke-lab-vpn-gateway VpnGw1AZ	P2S gateway

Source of truth: these are the implemented project ranges. Earlier concept diagrams with /24 hub subnets are not deployment values.

Traffic flow

Laptop -> Azure VPN Client/P2S -> VPN Gateway -> Hub VNet
-> VNet peering -> Spoke 1 Windows VMSS / IIS / .NET
-> VNet peering -> Spoke 2 Ubuntu / Flask 10.2.1.4:5000
Applications -> Microsoft Entra ID for interactive authentication

2. Azure Service Definitions

VNet: private Layer-3 network boundary. All project address spaces are non-overlapping so peering and VPN routing work.

GatewaySubnet: reserved for Azure Virtual Network Gateway. Project range: 10.0.1.0/27. Do not deploy ordinary VMs into it.

AzureFirewallSubnet: reserved for Azure Firewall. Project range: 10.0.2.0/26.

VNet peering: links hub and spokes over the Azure backbone. Gateway transit was enabled so spoke traffic can use the hub VPN gateway.

Windows VMSS: Windows Server 2022 scale-set workload in 10.1.1.0/24. IIS hosts the .NET Employee Portal. Autoscale was configured with a 1-3 instance range.

Linux VM: Ubuntu 22.04.5 LTS, hostname linuxapp01, private IP 10.2.1.4. Flask listens on TCP/5000.

Microsoft Entra ID: identity provider. Windows uses Microsoft.Identity.Web/OIDC. Linux uses MSAL Python authorization-code flow. MFA depends on Entra policy.

Azure Monitor/Log Analytics: monitoring design for VM/app logs, alerts and audit visibility. Keep lab retention modest to control cost.

3. Prerequisites and Tools

- Azure subscription and permissions for network, security, compute and monitoring resources.
- Terraform, Azure CLI and PowerShell on Windows.
- Azure VPN Client.
- PuTTY + PuTTYgen.
- .NET SDK for Employee Portal build/publish.
- Browser access to Azure Portal / Microsoft Entra.

Working directory

D:\Rajan\Cloud Project\Azure Hub-Spoke Architecture\terraform

Initial checks

```
az --version
az login
az account show
terraform version
terraform init
terraform fmt -recursive
terraform validate
terraform plan
```

Never commit Terraform state, client secrets, .env files, PFX files, or SSH private keys to a public repository.

4. Terraform Deployment Workflow

Keep the original project .tf files as the authoritative infrastructure definition. A maintainable layout is shown below.

```
terraform/
  providers.tf
  variables.tf
  resource-group.tf
  hub-network.tf
  spoke1-network.tf
  spoke2-network.tf
  peering.tf
  firewall.tf
  routing.tf
  windows-vmss.tf
  linux-vm.tf
  vpn-gateway.tf
  monitoring.tf
  outputs.tf
  id_rsa.pub
```

Core network values

Hub VNet:	10.0.0.0/16
GatewaySubnet:	10.0.1.0/27
AzureFirewallSubnet:	10.0.2.0/26
Spoke 1:	10.1.0.0/16
vmss-subnet:	10.1.1.0/24
Spoke 2:	10.2.0.0/16
linux-app-subnet:	10.2.1.0/24

Reference Terraform pattern

```
resource "azurerm_virtual_network" "hub" {
  name                = "hub-spoke-lab-hub-vnet"
  address_space       = ["10.0.0.0/16"]
  location             = azurerm_resource_group.lab.location
  resource_group_name = azurerm_resource_group.lab.name
}

resource "azurerm_subnet" "gateway" {
  name                 = "GatewaySubnet"
  address_prefixes     = ["10.0.1.0/27"]
  virtual_network_name = azurerm_virtual_network.hub.name
  resource_group_name  = azurerm_resource_group.lab.name
}

resource "azurerm_subnet" "firewall" {
  name                 = "AzureFirewallSubnet"
  address_prefixes     = ["10.0.2.0/26"]
  virtual_network_name = azurerm_virtual_network.hub.name
  resource_group_name  = azurerm_resource_group.lab.name
}
```

Apply sequence

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out tfplan
terraform apply tfplan
terraform output
terraform state list
```

A later gateway-transit update completed with 0 added, 2 changed, 0 destroyed. Refresh the VPN profile after routing/gateway changes.

5. Point-to-Site VPN

Gateway: hub-spoke-lab-vpn-gateway; SKU: VpnGw1AZ; P2S pool: 172.16.100.0/24. The client profile carried routes for 10.0.0.0/16, 10.1.0.0/16 and 10.2.0.0/16.

Certificate-authentication pattern

```
$root = New-SelfSignedCertificate `
  -Type Custom -KeySpec Signature `
  -Subject "CN=HubSpokeP2SRoot" `
  -KeyExportPolicy Exportable -HashAlgorithm sha256 `
  -KeyLength 2048 -CertStoreLocation "Cert:\CurrentUser\My" `
  -KeyUsageProperty Sign -KeyUsage CertSign

$client = New-SelfSignedCertificate `
  -Type Custom -DnsName "HubSpokeP2SClient" `
  -KeySpec Signature -Subject "CN=HubSpokeP2SClient" `
  -KeyExportPolicy Exportable -HashAlgorithm sha256 `
  -KeyLength 2048 -CertStoreLocation "Cert:\CurrentUser\My" `
  -Signer $root
```

Export the root public certificate for Azure and install/export the client certificate with private key only on authorized client machines. Protect PFX/private keys.

Connectivity test from laptop

```
Test-NetConnection 10.2.1.4 -Port 22
Test-NetConnection 10.2.1.4 -Port 5000

# Observed successful path:
# InterfaceAlias      : Azure-Hub-Spoke-P2S
# SourceAddress       : 172.16.100.2
# TcpTestSucceeded    : True
```

Azure VPN Client profiles may not appear as classic Windows phone-book entries to Get-VpnConnection. Validate using the VPN client, routes and actual private connectivity.

6. Firewall, Routing and NSGs

Azure Firewall resides in AzureFirewallSubnet. The project design also used NSGs and UDRs so workload traffic could be controlled centrally. Application VMs were kept private.

Typical spoke UDR:

Destination: 0.0.0.0/0

Next hop: Virtual appliance

Next hop IP: Azure Firewall private IP

Use only after firewall policy/rules are ready.

A bad UDR can break package downloads, Entra sign-in or application traffic.

- Linux admin: TCP/22 only from trusted VPN/client ranges.
- Linux Flask lab: TCP/5000 only where required.
- Windows/IIS: expose only intended HTTP/HTTPS bindings.
- Avoid public IPs on application VMs when P2S/private administration is available.

7. Spoke 1 - Windows VMSS, IIS and .NET

Spoke 1 uses Windows Server 2022 VMSS in vmss-subnet 10.1.1.0/24. The Employee Portal is ASP.NET Core/.NET hosted through IIS.

Application directory

C:\WebApps\EmployeePortal

Packages confirmed

```
Target framework: net10.0
Microsoft.Identity.Web 4.15.0
Microsoft.Identity.Web.UI 4.15.0
```

Build/publish

```
cd C:\WebApps\EmployeePortal
dotnet restore
dotnet build
dotnet list package
dotnet publish -c Release -o C:\WebApps\EmployeePortal-Publish
```

The recorded build completed with 0 warnings and 0 errors.

IIS deployment pattern

```
Import-Module WebAdministration
Stop-WebAppPool -Name "EmployeePortal"
```

```
Remove-Item "C:\inetpub\EmployeePortal\*" -Recurse -Force
Copy-Item "C:\WebApps\EmployeePortal-Publish\*" `
  "C:\inetpub\EmployeePortal\" -Recurse -Force
```

```
Start-WebAppPool -Name "EmployeePortal"
iisreset
```

If EmployeePortal.dll is locked, stop the IIS app pool/site before replacing published files. That exact lock was encountered in the project.

- Install IIS and the ASP.NET Core Hosting Bundle matching the runtime.
- Use an IIS app pool suitable for ASP.NET Core; 'No Managed Code' is the normal pattern.
- For VMSS, automate IIS/app deployment so every scale-out instance is identical.

8. Spoke 1 - Microsoft Entra ID

The Windows portal used a Web app registration, OpenID Connect and Microsoft.Identity.Web.

appsettings.json pattern

```
{
  "AzureAd": {
    "Instance": "https://login.microsoftonline.com/",
    "TenantId": "<TENANT-ID>",
    "ClientId": "<CLIENT-ID>",
    "CallbackPath": "/signin-oidc"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Program.cs pattern

```
using Microsoft.AspNetCore.Authentication.OpenIdConnect;
using Microsoft.Identity.Web;
using Microsoft.Identity.Web.UI;

var builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddAuthentication(OpenIdConnectDefaults.AuthenticationScheme)
    .AddMicrosoftIdentityWebApp(builder.Configuration.GetSection("AzureAd"));

builder.Services.AddControllersWithViews().AddMicrosoftIdentityUI();

var app = builder.Build();
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapDefaultControllerRoute();
app.MapRazorPages();
app.Run();
```

Redirect URIs used/attempted

Local development:
`http://localhost:5257/signin-oidc`

Private custom-name target attempted:
`https://employeeportal.hubspoke.test/signin-oidc`

Errors and fixes

AADSTS700054: response_type 'id_token' is not enabled
AADSTS50011: redirect URI does not match configured URI

- Use Web platform for server-side ASP.NET Core.
- Register the exact scheme + host + port + path sent by the app.
- Do not use implicit-flow settings as a random workaround for a redirect mismatch.
- The laptop custom-hostname path failed DNS resolution; it was not needed for the core learning objective.

```
Resolve-DnsName employeeportal.hubspoke.test
Test-NetConnection employeeportal.hubspoke.test -Port 443
Get-Content C:\Windows\System32\drivers\etc\hosts
ipconfig /flushdns
```

9. Spoke 2 - Ubuntu and Flask

VM: Ubuntu 22.04.5 LTS, hostname linuxapp01, private IP 10.2.1.4. Flask listens on 0.0.0.0:5000.

Discovery/validation

```
hostname
cat /etc/os-release
python3 --version
sudo ss -tulpn
curl http://localhost:5000
curl http://10.2.1.4:5000
ps -fp <PID>
sudo readlink -f /proc/<PID>/cwd
sudo tr '\0' ' ' < /proc/<PID>/cmdline
echo
```

Observed values

```
Python: 3.10.12
SSH:    0.0.0.0:22
Flask:  0.0.0.0:5000
App:    /opt/flaskapp/app.py
```

Health endpoint

```
@app.route("/health")
def health():
    return {
        "status": "healthy",
        "server": socket.gethostname(),
        "spoke": "spoke-2"
    }
```

10. PuTTY / SSH Access

Terraform referenced `id_rsa.pub`, but the matching original private key was not present on the laptop. Native SSH therefore returned Permission denied (publickey).

```
Select-String -Path *.tf `
  -Pattern "admin_ssh_key","public_key","tls_private_key","private_key"

# linux-vm.tf:
# public_key = file("${path.module}/id_rsa.pub")
```

Recovery used

- Generate a new RSA key pair in PuTTYgen.
- Save public key and matching private .ppk securely.
- Azure Portal -> VM -> Reset password or keys -> Add SSH public key.
- Username: azureadmin; paste the new public key and update.
- PuTTY -> host 10.2.1.4 -> port 22 -> SSH -> Auth/Credentials -> select matching .ppk.
- Connect while P2S VPN is active.

```
Test-NetConnection 10.2.1.4 -Port 22
# Expected: TcpTestSucceeded : True
```

Test-NetConnection is a PowerShell command. On Ubuntu use curl, ss, nc or other Linux-native tools.

11. Spoke 2 - Entra ID / MSAL

Back up the app and create a dedicated Python virtual environment.

```
sudo cp /opt/flaskapp/app.py /opt/flaskapp/app.py.backup
cd /opt/flaskapp
sudo apt update
sudo apt install -y python3.10-venv
sudo rm -rf /opt/flaskapp/.venv
sudo python3 -m venv /opt/flaskapp/.venv
sudo chown -R azureadmin:azureadmin /opt/flaskapp/.venv
source /opt/flaskapp/.venv/bin/activate
```

Packages

```
pip install --upgrade pip
pip install Flask msal python-dotenv requests
pip list | grep -Ei "Flask|msal|dotenv|requests"
```

```
# Project versions:
# Flask 3.1.3
# msal 1.39.0
# python-dotenv 1.2.3
# requests 2.34.2
```

Entra app registration

- Name: Linux Employee Portal.
- Single-tenant organizational directory.
- Platform: Web.
- Redirect URI: `http://localhost:5000/getAToken`.
- Create a client secret for the lab and copy the secret VALUE.

A client secret appeared in screenshots during the lab. Any exposed secret must be revoked and replaced. Never reuse or publish it.

.env

```
TENANT_ID=<YOUR-TENANT-ID>
CLIENT_ID=<YOUR-CLIENT-ID>
CLIENT_SECRET=<YOUR-ROTATED-SECRET>
REDIRECT_PATH=/getAToken

sudo chown azureadmin:azureadmin /opt/flaskapp/.env
chmod 600 /opt/flaskapp/.env
```

Verify variables without printing values

```
python -c "from dotenv import load_dotenv; import os; load_dotenv(); print(
'Tenant:', bool(os.getenv('TENANT_ID'))),
'Client:', bool(os.getenv('CLIENT_ID'))),
'Secret:', bool(os.getenv('CLIENT_SECRET')))"
```

Flask/MSAL reference implementation

```
import os, socket, secrets, msal
from flask import Flask, redirect, request, session, url_for
from dotenv import load_dotenv

load_dotenv("/opt/flaskapp/.env")
app = Flask(__name__)
app.secret_key = secrets.token_hex(32)

TENANT_ID=os.getenv("TENANT_ID")
CLIENT_ID=os.getenv("CLIENT_ID")
CLIENT_SECRET=os.getenv("CLIENT_SECRET")
AUTHORITY=f"https://login.microsoftonline.com/{TENANT_ID}"
REDIRECT_URI="http://localhost:5000/getAToken"

def build_msal_app():
    return msal.ConfidentialClientApplication(
        CLIENT_ID, authority=AUTHORITY, client_credential=CLIENT_SECRET)

@app.route("/")
def home():
    if session.get("user"):
        return '<a href="/dashboard">Employee Dashboard</a>'
    return '<a href="/login">Login with Microsoft Entra ID</a>'

@app.route("/login")
def login():
    flow=build_msal_app().initiate_auth_code_flow(
        scopes=[], redirect_uri=REDIRECT_URI)
    session["flow"]=flow
    return redirect(flow["auth_uri"])

@app.route("/getAToken")
def authorized():
    try:
        result=build_msal_app().acquire_token_by_auth_code_flow(
            session.get("flow",{}), request.args)
    except ValueError:
        return "Authentication state validation failed.",400
    if "error" in result:
        return result.get("error_description","Authentication failed"),400
    session["user"]=result.get("id_token_claims")
    return redirect(url_for("dashboard"))

@app.route("/dashboard")
def dashboard():
    user=session.get("user")
    if not user: return redirect(url_for("login"))
    return f"<h1>Employee Dashboard</h1><p>Welcome {user.get('name','Employee')}</p>"

@app.route("/logout")
def logout():
    session.clear()
    return redirect(
        f"https://login.microsoftonline.com/{TENANT_ID}/oauth2/v2.0/logout"
        "?post_logout_redirect_uri=http://localhost:5000/")

@app.route("/health")
def health():
    return {"status":"healthy","server":socket.gethostname(),"spoke":"spoke-2"}

if __name__=="__main__":
    app.run(host="0.0.0.0",port=5000,debug=False)
```

Syntax + systemd

```
/opt/flaskapp/.venv/bin/python -m py_compile /opt/flaskapp/app.py

# /etc/systemd/system/flaskapp.service
[Unit]
Description=Azure Hub Spoke Flask Application
After=network.target

[Service]
User=root
WorkingDirectory=/opt/flaskapp
ExecStart=/opt/flaskapp/.venv/bin/python /opt/flaskapp/app.py
Restart=always
```

```
[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl restart flaskapp
sudo systemctl status flaskapp --no-pager
curl http://localhost:5000/health
curl http://localhost:5000/
curl -I http://localhost:5000/login
```

Recorded result: flaskapp.service was active and used /opt/flaskapp/.venv/bin/python; /health returned healthy.

PuTTY local tunnel for the Linux callback

```
PuTTY -> Connection -> SSH -> Tunnels
Source port: 5000
Destination: 127.0.0.1:5000
Select: Local + Auto
Add -> L5000 127.0.0.1:5000

# On Windows laptop:
Test-NetConnection localhost -Port 5000

# Browser:
http://localhost:5000
```

Flow: laptop localhost:5000 -> PuTTY tunnel -> SSH over P2S -> linuxapp01:5000 -> /login -> Entra -> MFA when policy requires -> /getAToken -> protected dashboard.

12. End-to-End Testing

Network

```
# Windows
Test-NetConnection 10.2.1.4 -Port 22
Test-NetConnection 10.2.1.4 -Port 5000

# Linux
sudo ss -tulpn
curl http://localhost:5000/health
curl http://10.2.1.4:5000
```

Windows app

```
cd C:\WebApps\EmployeePortal
dotnet build
dotnet list package
# Local browser during development:
http://localhost:5257
```

Identity checklist

- Web platform selected in Entra.
- Redirect URI exactly matches application request.
- Protected page redirects unauthenticated users to sign-in.
- Successful sign-in creates local application session.
- Logout clears local session.
- MFA is tested only for users/policies where Entra requires it.

Linux logs

```
sudo journalctl -u flaskapp -n 50 --no-pager
sudo journalctl -u flaskapp -f
```

13. Troubleshooting Lessons

Symptom	Cause	Resolution
AADSTS50011	Redirect URI mismatch	Register exact scheme/host/port/path.
AADSTS700054	Token response/config mismatch	Use correct Web/OIDC authorization-code setup.
DNS name does not exist	Custom name not resolvable on laptop	Fix DNS/hosts or avoid custom name for lab.
DLL access denied	IIS holds EmployeePortal.dll	Stop app pool/site before copying files.
Permission denied (publickey)	Missing matching SSH private key	Create new pair; add public key; use matching .ppk.
venv creation failed	python3.10-venv missing	Install package; delete incomplete .venv; recreate.
Test-NetConnection not found	PowerShell command run in Bash	Run on Windows; use curl/ss/nc on Linux.
MSAL import would fail after restart	systemd used /usr/bin/python3	Point ExecStart to .venv/bin/python.
Get-VpnConnection profile missing	Azure VPN Client differs from classic VPN entry	Validate via Azure VPN Client/routes/connectivity.

14. Operations and Security Improvements

- Use Unicorn + Nginx (or another production web tier) instead of Flask's development server.
- Run Linux application under a dedicated non-root service account.
- Use Azure Key Vault / managed identity where practical instead of long-lived client secrets.
- Use HTTPS, production DNS and valid certificates.
- Automate VMSS application provisioning so all scale-out instances are identical.
- Create Azure Monitor alerts for VM, VPN Gateway, Firewall and application health.
- Use least-privilege NSGs/firewall rules and document every required port.
- Control Log Analytics retention to avoid unnecessary lab cost.

Client secrets/private keys are credentials. Any credential shown in a screenshot, chat, ticket or repository should be revoked and replaced.

15. Cost Control and Teardown

The largest lab cost drivers are typically always-on managed/network services and compute. VPN Gateway and Azure Firewall can continue accruing charges even when the lab is idle; VMSS, Linux VM, public IPs and monitoring can also cost money.

Preserve before deleting

- Terraform .tf files and non-secret variable templates.
- Application source and deployment instructions.
- Architecture/runbook/screenshots for portfolio evidence.
- Do not preserve exposed/expired secrets in documentation.

Terraform teardown

```
cd "D:\Rajan\Cloud Project\Azure Hub-Spoke Architecture\terraform"
terraform plan -destroy
# Review carefully.
terraform destroy
# Type yes only after review.
```

- After destroy, check Azure Portal for resources created manually outside Terraform.
- Verify VPN Gateway, Azure Firewall/public IP, VMSS, Linux VM/disks, route tables, NSGs and monitoring resources are gone if they belong only to this lab.
- Cost records can appear with delay; review Cost Management after teardown.

Keeping code/Terraform while destroying deployed resources is the low-cost portfolio approach.

16. Clean Redeployment Checklist

1. Install Azure CLI, Terraform, .NET SDK, Azure VPN Client, PuTTY/PuTTYgen.
2. Authenticate to Azure and select the intended subscription.
3. Initialize Terraform and deploy resource group.
4. Deploy Hub VNet 10.0.0.0/16.
5. Create GatewaySubnet 10.0.1.0/27 and AzureFirewallSubnet 10.0.2.0/26.
6. Deploy Spoke 1 10.1.0.0/16 + vmss-subnet 10.1.1.0/24.
7. Deploy Spoke 2 10.2.0.0/16 + linux-app-subnet 10.2.1.0/24.
8. Create hub/spoke peerings and gateway transit.
9. Deploy Azure Firewall, NSGs and intended UDRs.
10. Deploy VpnGw1AZ and configure P2S pool 172.16.100.0/24 + certificate authentication.
11. Import VPN profile and validate private routes.
12. Deploy Windows Server 2022 VMSS; configure autoscale 1-3.
13. Install IIS/ASP.NET Core hosting requirements and publish Employee Portal.
14. Create Windows Entra Web app registration; add exact /signin-oidc redirect URI; validate sign-in.
15. Deploy Ubuntu 22.04.5 VM in Spoke 2.
16. Configure SSH key access; validate TCP/22 over VPN.
17. Deploy Flask app under /opt/flaskapp; validate /health on 5000.
18. Create .env; install Flask/MSAL/python-dotenv/requests.
19. Create Linux Entra Web app registration with `http://localhost:5000/getAToken`.
20. Secure .env; update app.py for MSAL authorization-code flow.
21. Update flaskapp.service to .venv Python; daemon-reload/restart.
22. Validate local app, VPN access and PuTTY local tunnel.
23. Capture evidence and destroy the lab when no longer needed.

17. Command Reference

Terraform

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan
terraform apply
terraform output
terraform state list
terraform plan -destroy
terraform destroy
```

Windows / VPN

```
Test-NetConnection 10.2.1.4 -Port 22
Test-NetConnection 10.2.1.4 -Port 5000
Resolve-DnsName employeeportal.hubspoke.test
route print
ipconfig
ipconfig /flushdns
```

.NET

```
dotnet restore
dotnet build
dotnet list package
dotnet publish -c Release -o C:\WebApps\EmployeePortal-Publish
```

Linux

```
hostname
cat /etc/os-release
python3 --version
sudo ss -tulpn
curl http://localhost:5000/health
sudo systemctl status flaskapp --no-pager
sudo journalctl -u flaskapp -n 50 --no-pager
```

Python

```
sudo apt install -y python3.10-venv
python3 -m venv /opt/flaskapp/.venv
source /opt/flaskapp/.venv/bin/activate
pip install Flask msal python-dotenv requests
python -m py_compile /opt/flaskapp/app.py
```

Closing note

This runbook separates the implemented lab from optional production hardening. The completed learning architecture is a Central India hub-and-spoke environment with P2S VPN, Azure Firewall, a Windows VMSS/.NET/IIS spoke and an Ubuntu/Flask spoke, with Microsoft Entra ID authentication patterns demonstrated across both stacks.